Suitunes — Real-Time Pitch-Correction Engineering Specification

Research reference / V1 repair plan
Date: November 5, 2026
Status: Research findings and proposed engineering acceptance targets, **not** audited V1 source code or proven Pitcher parity.

Executive summary

Suitunes is a real-time **monophonic vocal pitch corrector**, not merely a generic pitch shifter. In V1, setting G minor produced audibly wrong results compared with FL Studio Pitcher on the same recording. The root cause is **not yet established**: it could be fundamental-frequency (F0) detection, scale/musical-note mapping, target-note stability, pitch-ratio calculation, or audio resynthesis.

Do not rewrite the V1 engine without evidence. Preserve the current build, add traceable diagnostics, test the pitch decision path without audio shifting, then isolate synthesis quality. Image-Line's Pitcher algorithm is proprietary; the target is comparable observable and audible behavior rather than a claim to duplicate its internals.

Recommended modular chain:

```
Vocal input → voicing/onset detector + multi-candidate F0 analysis → pitch-state/octave tracker → key/scale note mapper → correction trajectory (speed/strength/vibrato) → formant-aware resynthesis → output
```

For a prototype, evaluate an independently implemented multi-candidate YIN-style detector and state tracker alongside **Signalsmith Stretch** as a replaceable pitch-shifting backend. Independently evaluate **TD-PSOLA** for a more vocal-specific future backend. No method guarantees Pitcher-quality sound before measurements and listening comparisons.

---

1. Pitch detection

Fundamental frequency and confidence

- Use a rolling analysis buffer and YIN-style difference function, cumulative-mean normalization, local-minimum extraction and parabolic interpolation.

- Retain **multiple F0 candidates**, each with confidence and harmonic-consistency information; do not immediately snap to one estimate per frame. This is informed by probabilistic YIN (pYIN), although full offline Viterbi decoding is not a drop-in low-latency solution.
- Maintain a persistent candidate-state tracker: favor plausible pitch continuity while allowing genuine note onsets and large intended interval jumps.
- Check candidate alternatives at `f/2` and `2f` to reduce octave mistakes, with calibrated penalties, vocalist range bounds and voicing-confidence gates.
- Measure energy, periodicity and spectral flatness to distinguish voiced vowels from breath, consonants, sibilance and silence. Avoid strong retuning during unvoiced portions.
- Use onset detection to distinguish a new note from pitch-tracker jitter. Hold or gently release correction when confidence is inadequate instead of inventing a target.
- Treat vibrato as a faster modulation around a slower note center, not necessarily a series of separate notes.

Critical V1 investigation

Inspect sample-rate handling, window sizes, low-note minimum frequency, boundary candidates, octave flipping, unknown/silent frames, pitch conversion, and whether processing depends on audio block size. Test spoken/sung low notes, breathy vocals, scoops, slides, vibrato, quick note changes and consonant-heavy phrases.

Detection regression metrics: median and 95th-percentile absolute cents error on reference F0; >50-cent gross error rate; octave-error rate; voiced/unvoiced precision and recall. Report these separately by vocal type and pitch range.

Research: YIN; pYIN; Praat/autocorrelation.

---

2. Correction algorithms

Math and scale mapping

At a reference tuning of A4=440 Hz (user-adjustable):

`m = 69 + 12 × log2(f0 / 440)` (continuous MIDI note value)

Allowed targets are integers whose pitch class belongs to the selected musical scale or custom mask. **G natural minor = G, A, B-flat, C, D, E-flat, F**, i.e. pitch classes `{7,9,10,0,2,3,5}` when C=0.

For a selected target MIDI note n:

```
correction_cents = 100 × (n − m)
pitch_ratio = 2^(strength × correction_cents / 1200)
```

Here `strength` is a continuous factor from 0 to 1; implementation must decide how the user-facing Weak / Medium / Strong settings map to it. Verify **shift direction** experimentally: when input is below target, output must rise in pitch.

Important mapping pitfalls:

- Consistent C=0 chromatic indexing and correct modulo for negative numbers.
- Correct natural-minor semitone mask and root transposition.
- Separation of scientific MIDI note numbering from FL Studio's displayed octave naming.
- No out-of-scale target when a scale mask applies.
- No unintentional octaves or unit conversions in the shifter API.
- Compare target notes using cents distance rather than approximate integer-frequency shortcuts.

Stateful correction, transitions and expression

- Keep the selected note **persistent**. Use hysteresis / minimum dwell before switching to a neighboring note; no rapid switching on noisy semitone boundaries.
- Map **Speed** to a time constant in milliseconds, not a fixed per-block interpolation coefficient. For block/hop Δt, a basic smoother is `y_new = y_old + [1 − exp(−Δt/τ)](target − y_old)`.
- Differentiate new-note attack, stable held note, legato transition, intentional portamento, vibrato and low-confidence release. A single global smoothing rule is insufficient.
- Apply Strength to the desired **pitch-error correction**; relying on a wet/dry blend alone can make the output phasey without producing the desired corrected fundamental.
- Optionally use a **capture zone**: don't force already-near-target notes to center.
- Preserve intentional vibrato by correcting slow note-center drift while retaining a controlled amount of faster expressive modulation.
- Keep MIDI-controlled targets explicit; distinguish pitch-class mode from absolute MIDI-octave mode.

Deterministic G-minor test: for every possible input chromatic pitch, generate the expected enabled target and verify that no result targets G-sharp, B-natural, C-sharp, E-natural or F-sharp. Test

midpoint ties, octave edges and notes below the MIDI middle range. This test must pass **before** listening to shifted audio.

Research: Pitcher documentation and behavior; Dynamic Pitch Warping; expressive pitch contour preservation.

---

3. Real-time pitch-shifting technology

Pitch correction sound quality depends both on correct target-note decisions and on clean, continuously controlled resynthesis. Test these separately.

Engine	Why evaluate	Caveats and distribution considerations
Signalsmith Stretch	C++ real-time pitch/time processor, latency reporting, dynamic pitch changes, formant compensation	**MIT**; promising initial prototype, but latency and subjective vocal quality must be measured for the selected configuration
Custom TD-PSOLA	Designed for pitch-synchronous speech processing; potentially strong monophonic-vocal control	Greater engineering burden, especially reliable pitch marks, breathy/unvoiced sounds, rapid transitions and fallback strategies
Rubber Band	Established real-time streaming pitch/time processing	Open-source **GPL** or commercial licensing; real-time padding/delay must be handled
SoundTouch	Well-known pitch/time library	Streaming latency and formant quality make it less attractive for strict live-monitoring goals; check LGPL/commercial licensing

Engine	Why evaluate	Caveats and distribution considerations
Phase vocoder	Flexible spectral pitch/time manipulation	Needs phase locking, transient handling and spectral-envelope preservation to avoid smearing/metallic voice

Formants, phase and transient handling

- Pitch and vocal-tract resonances (formants) are separate perceptual properties: shifting harmonics without treating the spectral envelope can cause exaggerated ‘chipmunk’ or deepened-voice coloration.
- Evaluate shifter built-in formant correction against independent envelope analysis (cepstral/true-envelope, LPC/source-filter, or F0-adaptive estimates).
- Preserve phase coherence through synthesis, especially at changing pitch ratios; smooth correction-ratio commands to avoid zipper noise and clicks.
- Handle unvoiced consonants, transients and breaths distinctly from pitched vowel material. Processing an entire phrase as steady periodic audio is a common artifact source.
- Do controlled tests at 0, ±1, ±3, ±6 and ±12 semitones. The routine vocal-quality target should focus on modest shifts; extreme shifts may always sound more artificial.

Research: Signalsmith Stretch; Signalsmith source and license; original PSOLA work; Rubber Band licensing; WORLD speech analysis; spectral envelope paper.

---

4. Performance, JUCE and FL Studio

Distinguish three latency quantities

1. **Analysis confidence delay:** how long it takes to obtain a sufficiently reliable F0 estimate.
2. **Shifter algorithmic delay:** buffering or lookahead in the resynthesis stage.

3. **Live monitor end-to-end delay:** plugin + host buffer + audio interface input/output and driver overhead.

A suggested **≤10 ms plugin algorithmic-delay goal** is aspirational, not verified. At 80 Hz, one full period is 12.5 ms; very low notes inherently compete with extremely short analysis delay. Use measured **Live** and **Quality** operating points, with a sensible minimum-frequency range and reliable fallback behavior.

Implementation requirements

- C++ DSP core independent of JUCE types for offline unit tests; JUCE `AudioProcessor` as host adapter.
- Allocate ring buffers, FFT plans, candidate lists, shifter working memory and all temporary storage before audio callbacks.
- No filesystem writes, exceptions, network access, locks, or heap allocations in `processBlock()`.
- Use atomic parameter reads / real-time-safe parameter snapshots. Smooth parameter changes; don't serialize presets on audio thread.
- Support 44.1/48/88.2/96 kHz and varied or changing host block sizes; reset state cleanly on transport changes and plugin reinitialization.
- Report measured shifter latency through the VST/JUCE host mechanism to support plug-in delay compensation.
- Keep debug traces in a lock-free bounded queue drained on a non-audio thread; avoid logging synchronously in live processing.
- Verify VST3 note/MIDI routing, automation, bypass, offline export, and user-reported CPU load in FL Studio.
- Confirm appropriate licensing for distributed JUCE versions and all dependencies. **Do not bundle GPL-licensed aubio code** inside a proprietary binary without resolving the license. Research published algorithms and implement independently where needed.

Research: JUCE AudioProcessor; JUCE parameter state; VST3 automation; JUCE commercial licensing.

---

5. Verification and direct comparison against Pitcher

Save the V1 baseline first

The V1 code was on the user's local PC and had **not yet been pushed** to `ajo6522/suitunes` at the time of research. No source audit has occurred. Tag the unmodified baseline before changing code.

Diagnostic trace (offline to CSV)

For each short analysis hop, capture:

```
sample_position, rms, voiced_confidence, onset_score,
candidate_f0_hz[], candidate_confidence[], chosen_f0_hz,
detected_midi_continuous, octave_rejection_reason,
key, scale_mask, target_midi, correction_cents,
shift_ratio, formant_setting, engine_latency_samples
```

Also measure the **actual corrected output F0** independently from the exported audio. When the output is wrong, compare: detected pitch → target pitch → requested ratio → measured output. This isolates detector, note-mapper, controller, and shifter faults.

Baseline test corpus

Use identical dry mono WAV stems: held G-minor notes; chromatic tests across and outside the mask; sustained vibrato; low male voice; high voice; breathy notes; consonants; pitch scoops; rapid note transitions; silence and background noise. Include synthetic known-F0 harmonic signals and a set of manually annotated natural vocal excerpts.

Required tests and measurements

1. **Detector-only:** Known-F0 sine/harmonic synthetic input and labeled real vocals; cents error, octave error, voiced/unvoiced classification.
2. **Scale mapper-only:** Every G-minor pitch class and all 12 chromatic inputs across several octaves. Assert targets never use disabled notes.
3. **Ratio direction:** Explicitly request +100/-100/+1200/-1200 cents; measure output and sign.
4. **Stable target state:** No note chatter on held notes with tiny frequency fluctuations around note boundaries; correct behavior at actual note onsets.

5. **Correction envelope:** Graph detected F0, chosen target and measured output F0 vs. time for different speed/strength values.
6. **Artifact/voice quality:** A/B with formants on/off, vibrato retained/reduced, attacks and unvoiced consonants; listening plus objective spectrogram and residual analysis.
7. **Latency:** Compare active and bypass renders via cross-correlation, confirm declared plugin latency, measure live roundtrip separately.
8. **Real-time safety and CPU:** Benchmark Release builds under variable block sizes and sample rates; measure average, 95th-percentile and worst callback durations; fail on missed audio deadlines.
9. **FL Studio / Pitcher reference:** Render the **same dry WAV** through Pitcher and Suitunes, document note mask, tuning reference, speed, formant mode, wet/dry and gain. Since settings may not have exact equivalents, record behavior and avoid pretending the parameter values are identical.
10. **Blind listening:** Randomized comparisons by vocal type; identify reproducible cases where Pitcher sounds better and correlate them with diagnostic plots.

Proposed acceptance gates — engineering targets, not observed results

Dimension	Initial acceptance goal
Clean monophonic target-note assignment	≥99% on curated reference phrases
Median voiced pitch detection error	≤5 cents
95th-percentile voiced pitch detection error	≤20 cents
Octave errors	<0.5% of voiced reference frames
Out-of-scale notes in G natural minor	**0** due to scale-mapping logic
Held-note stability	No rapid alternating targets on stable vocals
Plugin algorithmic latency	Aim for ≤10 ms where physically and musically practical; otherwise document tradeoff
Real-time audio safety	No blocking operations in audio callback; no audio dropouts under tested conditions

Dimension	Initial acceptance goal
Audible quality	Comparable to Pitcher in matched-input blind listening tests; **not yet demonstrated**

Prioritized repair workflow

Gate A — Detection correctness. Instrument V1 with tracing. Check Hz / MIDI conversion and octave errors using dry audio. Do not shift audio yet.

Gate B — Musical target correctness. Verify G-minor note masks, scale mapping, selection/hysteresis, reference pitch, ratio direction and transition state in deterministic tests.

Gate C — Correction trajectory correctness. Produce plots of source cents → intended target → smoothed ratio for identical recordings and settings; isolate vibrato/onset mistakes.

Gate D — Resynthesis comparison. Only after A–C pass, compare the existing shifter against Signalsmith and a PSOLA prototype using the **same target trajectory**.

Gate E — FL Studio parity. Validate audible quality, latency, CPU and DAW stability with actual vocal recordings and a direct Pitcher comparison.

V1 code-audit checklist for GitHub

Module / behavior	Audit questions
Hz ↔ MIDI conversion	Is A4=440 scaling and log2 conversion correct?
Scale masks	Is G natural minor {G,A,Bb,C,D,Eb,F}? Correct root offset/modulo?
MIDI octave	Is internal MIDI numbering independent of FL Studio display names?
F0 detector	Candidate confidence? Harmonic ambiguity? Low range?
Octave tracker	Rejects unsupported halving/doubling during sustained vowels?

Module / behavior	Audit questions
Voicing/onset	Stops retuning unvoiced consonants and unstable attacks?
State/hysteresis	Prevents adjacent-note chattering near thresholds?
Speed/strength	Correct units and smoothing? Strength acts on cents, not wet/dry alone?
Ratio direction	`2^(cents/1200)` passed with correct orientation?
Shifter	Reports latency? Supports smooth, varying ratio?
Formants	Real envelope processing rather than cosmetic EQ?
Audio processing	No allocations/locks/logging? Handles sample rate/block size changes?
FL Studio	Correct VST latency, output channel handling, MIDI and bypass behavior?
Regression tests	Detector, target mapper, output pitch and direct Pitcher renders present?

Key research references

1. Image-Line — Pitcher manual
2. de Cheveigné & Kawahara — YIN fundamental frequency estimator
3. Mauch & Dixon — pYIN
4. Praat — Pitch analysis by autocorrelation
5. Signalsmith Stretch: documentation
6. Signalsmith Stretch: code and MIT license
7. ISCA — Pitch-synchronous waveform processing
8. DAFx — Dynamic Pitch Warping for Expressive Vocal Retuning
9. DAFx — Spectral envelope estimation and pitch shifting
10. WORLD speech synthesis research
11. Rubber Band library and licensing
12. SoundTouch
13. JUCE documentation
14. VST3 developer portal — automation
15. JUCE license

Source notes: This document reorganizes and preserves the sub-

stantive conclusions from the completed Suitunes Deep Research report and its linked research sources. The reported quantitative thresholds are *proposed targets*, not measured performance. Library license statements must be rechecked against the exact dependency versions before commercial distribution. This is the pitch-correction repair reference; the broader Suitunes product specification (Auto Tune, Fine Tune, selections/zoom, .sts presets) remains a separate feature scope and should not be mistaken for implemented code.